BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 2

DCA1210

OBJECT - ORIENTED PROGRAMMING USING C++

Unit – 1

Introduction to Object-Oriented Programming

TABLE OF CONTENTS

1. INTRODUCTION

In this unit, learners will learn about the programming that has evolved over time to handle the growing complexity of software development. Initially, procedural programming was the main approach, where tasks were broken down into a series of functions. However, as software projects became bigger and more complex, this method showed its limits, making it harder to maintain and expand programs. This led to the creation of Object-Oriented Programming (OOP), which organizes software around "objects"—small units that combine data and the actions that work on that data.

Leraners will also learn important concepts like encapsulation, inheritance, and polymorphism. Encapsulation protects an object's data from being changed accidentally by other parts of the program. Inheritance allows new classes to be built on top of existing ones, making it easier to reuse code. Polymorphism allows the same function to behave differently depending on the object it is used on. These features make OOP especially useful for developing large and complex software systems.

To effectively learn the topics in this unit, learners should start by mastering the basics of procedural programming, using hands-on practice and real-world analogies to grasp OOP concepts. Visualizing relationships between classes and objects, comparing C and C++ through coding exercises, and exploring real-world applications will reinforce their understanding.

1.1. Objectives

After studying this unit, you should be able to:

- Identify the key programming methodologies that have evolved over time, including procedural programming and object-oriented programming.
- Define the basic concepts of Object-Oriented Programming (OOP).
- List the key characteristics of OOP.
- List the primary differences between C and C++ in terms of programming paradigms, syntax, and features.

2. EVOLUTION OF PROGRAMMING METHODOLOGIES

Computer performs a variety of tasks with the help of programming languages. The first version of a programming language was the machine-level language. During the1940s, the machine-level language, which used the binary codes of 0s and 1s to represent computer instructions, was developed. Machine-level language is also known as low-level programing language. Writing programs in machine language is symbolic, and had to be error prone (which is impractical). So, assembly languages were developed in the early 1950s. Assembly languages use mnemonics to write the instruction of a program.

First high level programming language known as FORTRAN was developed in the year 1957. During the 1960s, many high level languages were developed to support the needs of various disciplines like science and engineering, and business. The period between 1970 and 1980 was actually the golden era for the development of high-level programming languages. During 1970, a procedural programming language named PASCAL was developed. The C programming language was developed by Dennis Ritchie at the Bell Labs during the year 1973. In 1980, Bjarne Stroustrup, from the Bell labs, began the development of the C++ language.

The idea of object-oriented programming gained momentum in the 1970s and in the early 1980s. Bjorn Stroustrup integrated the object-oriented programming into the C language. C++ is a superset of C. Several features are similar in C and C++. At this time, many new features like object, class,inheritance, virtual functions and function overloading were added. Therefore, there is a '++' symbol in C++. The ++ operator in the C++ means many new features were added around this time. The most notable features are - object, class, inheritance, virtual functions and function overloading.

Limitations of procedural languages:

Procedural languages focused on organizing program statements into procedures or functions. Larger programs were either broken into functions or modules which had defined purpose and interface to other functions.

Procedural approach for programming had several problems as the size of the software grew larger and larger. One of the main problems was that of the data being completely forgotten. The emphasis was on the action and the data was only used in the entire process. Data in the program was created by variables, and if more than one function had to access data, then global variables were used. The concept of global variables itself is a problem as it may be accidentally modified by an undesired

function. This also leads to difficulty in debugging and modifying the program when several functions access a particular data.

The object Oriented approach overcomes this problem by modeling data and functions together, thereby allowing only certain functions to access the required data.

The procedural languages had limitations of extensibility as there was limited support for creating user-defined data types and defining how these datatypes are to be handled. For example, it would be difficult if the programmer had to define his own version of string, and define how this new datatypes will be manipulated. The Object Oriented Programming provides this flexibility through the concept of class.

Another limitation of the procedural languages is that the program model is not closer to real world objects. For example, if you want to develop a gaming application of car race, what data you would use and what functions you would require are difficult questions to answer in the procedural approach. The object oriented approach solves this further by conceptualizing the problem as group of objects which have their own specific data and functionality. In the car game, for example we would create several objects such as player, car, and traffic signal and so on.

Some of the languages that use object oriented programming approach are C++, Java, Csharp, Smalltalk etc. We will be learning C++ in this Self Learning Material (SLM) to understand the object oriented programming.

3. INTRODUCTION TO OOP AND ITS BASIC FEATURES

OOP allows developers to decompose complex problems into smaller, more manageable entities called objects. Each object encapsulates both data and the functions that operate on that data, creating a self-contained unit. This decomposition makes the program more organized, as each object represents a specific aspect of the problem and can be developed and tested independently. Also, this approach promotes code reuse, as objects can be reused across different parts of the program or even in different programs.

OOP introduces a controlled interaction between objects, where an object's data can only be accessed by the functions within that object, preserving the integrity of the data. The design of OOP allows objects to interact with each other, where the functions of one object can call the functions of another, enabling collaboration between different parts of the program. This balance between encapsulation and inter-object communication is a cornerstone of OOP, making it a powerful paradigm for building complex, scalable, and maintainable software systems.

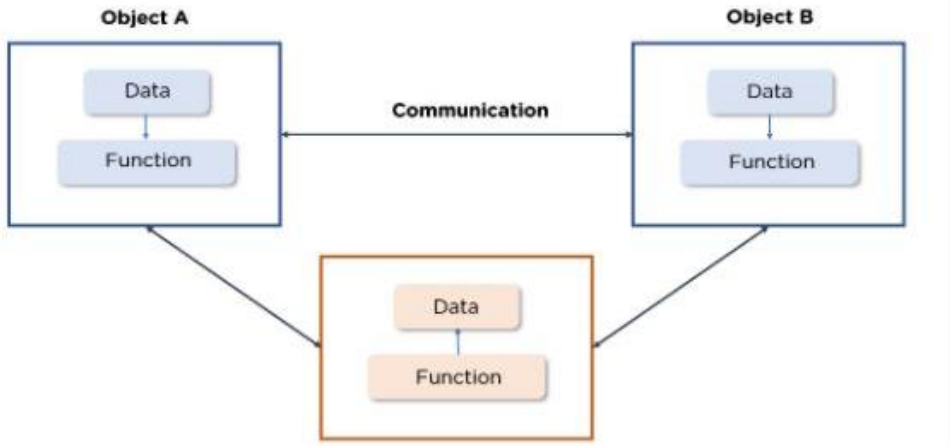

Figure 1: OOP's data and function organisation

There are several fundamental concepts that are extensively used in object-oriented programming:

- Objects
- Classes
- Data Abstraction and Encapsulation
- Inheritance
- Polymorphism
- Dynamic Binding
- Message Passing

Objects: Objects are described as the fundamental run-time entities in an object-oriented system, representing various elements such as a person, place, bank account, or any other item that the program needs to manage. They can also represent more abstract data types like vectors, time, and lists. The objects are chosen based on their relevance to real-world entities, ensuring that they closely mirror the actual items or concepts they are designed to represent.

Each object occupies memory space and has an associated address, similar to records in Pascal or structures in C. This emphasizes the tangible nature of objects in memory. The interaction between objects is also highlighted, where objects communicate by sending messages to each other.

For example, in a program with "customer" and "account" objects, the customer object might send a message to the account object to request the bank balance. This interaction allows objects to work together without needing to know the internal details of each other's data or code, focusing instead on the type of message accepted and the response returned by the objects.

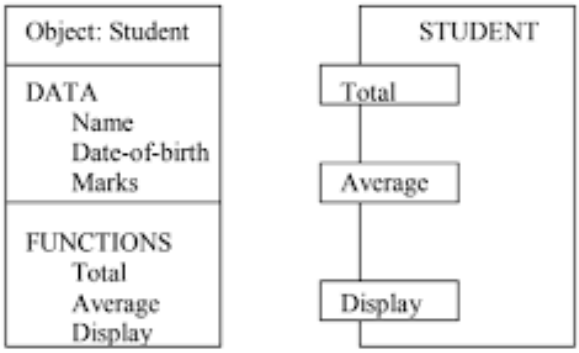

Figure 2: Representation of an object in two different ways

The figure 2 represents two common notations for representing objects, specifically using the example of a "STUDENT" object. The first notation lists the object's data (such as Name, Date-of-birth, and Marks) and functions (such as Total, Average, and Display) in a structured format, showing how the object encapsulates both data and behavior. The second notation simplifies the representation by focusing on the functions associated with the object, emphasizing the different ways objects can be visualized in object-oriented analysis and design.

Classes: A class is essentially a blueprint for creating objects. Once a class is defined, any number of objects can be created based on that class. These objects will share the same structure and behaviour as defined in the class, i.e., they will contain the same type of data and the same functions to manipulate that data (Hence class can be defined as a collection of similar objects). Objects are variables of the type class i.e., an object is an instance of a class, and it carries the characteristics (data

and behavior) defined by the class. For example, if there is a class named "fruit," then objects like "mango," "apple," and "orange" can be created as instances of the "fruit" class.

A class is described as a collection of objects of a similar type i.e., all objects created from a class share common attributes and functions.

For example, the class "fruit" may have attributes like color and taste, and functions like ripen or spoil, which all instances (such as mango, apple, orange) will have.

Classes are considered user-defined data types, which behave similarly to built-in types in a programming language i.e., just as you can create an integer or a float in C, you can create an object using the same kind of syntax once the class is defined.

For example, the statement ***fruit mango***; creates an object "mango" that belongs to the class "fruit."

If a class named "fruit" has been defined, then writing fruit mango; in a program will create an object named "mango," which is an instance of the class "fruit." This object will possess all the characteristics and behaviors defined in the "fruit" class.

Data Abstraction and **Encapsulation**: Encapsulation is the process of bundling data and functions that operate on that data into a single unit, known as a class. This technique ensures that the data within a class is not accessible directly from the outside world; instead, it can only be accessed or modified through the functions provided within the class. These functions serve as an interface between the object's data and the rest of the program. The purpose of encapsulation is to protect the data from unintended interference or misuse, a principle often referred to as **data hiding** or **information hiding** as shown in figure 3.

Figure 3: Encapsulation

In OOP, classes uses abstraction by defining objects through a list of abstract attributes (like size, weight, or cost) and corresponding functions that operate on these attributes.

Data abstraction is a principle in Object-Oriented Programming (OOP) that involves representing the essential features of an object while hiding the underlying implementation details. It allows programmers to focus on what an object does rather than how it does it.

By using data abstraction, you can define an interface that exposes only the necessary attributes and methods needed to interact with an object, while the complexity of how these attributes and methods are implemented is kept hidden from the user.

Data abstraction is achieved through the use of classes, which encapsulate data (attributes) and functions (methods) into a single unit. The internal workings of these functions are not visible to the outside world; instead, the user interacts with the object through a simplified interface that abstracts away the complexities.

For example, consider a "Car" class in a program. The user of this class might interact with it using methods like startEngine(), drive(), and stopEngine(). The user does not need to know the intricate details of how the engine is started or how the car moves—those details are abstracted away. The focus is on what the car can do (its behavior), not how it does it.

Data abstraction helps in managing complexity, improving code readability, and providing a clear separation between an object's external interface and its internal implementation.

By focusing on these abstract properties, abstraction allows for the creation of simplified representations of complex systems. The attributes are often referred to as **data members** because they store information, while the functions that manipulate these data members are called **methods** or **member functions**.

Since classes in OOP utilize data abstraction, they are often referred to as **Abstract Data Types (ADT)**. This term emphasizes that classes provide a high-level interface to the data and behaviors they encapsulate, abstracting away the details of their implementation. This combination of abstraction and encapsulation is a key aspect of OOP, enabling the creation of flexible, reusable, and maintainable code.

Inheritance : Inheritance is one of the fundamental principles of Object-Oriented Programming (OOP). It allows a new class, known as a **derived class** or **subclass**, to inherit properties and behaviors (attributes and methods) from an existing class, known as a **base class** or **superclass** (i.e., the process by which a class (or object) can acquire properties and behaviors from another class). This mechanism enables the creation of a class hierarchy where more specialized classes build on the general characteristics of more generic ones.

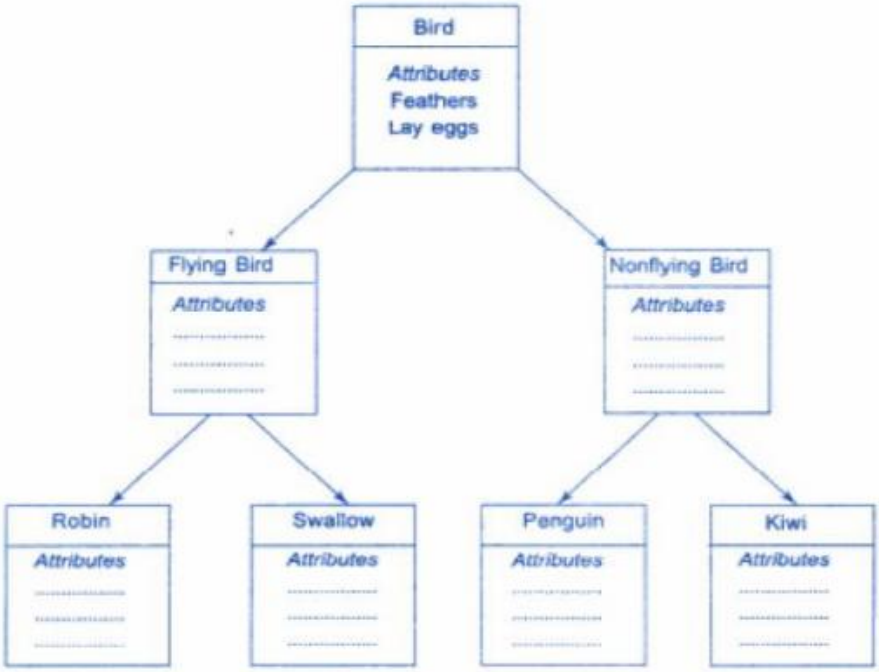

Figure 4: Inheritence of a property.

For example in figure 4, the "Bird" class represents a general category with common attributes such as "Feathers" and "Lay eggs." This class is then divided into two subclasses: "Flying Bird" and "Nonflying Bird," each representing a more specific category. The "Flying Bird" subclass might include birds like "Robin" and "Swallow," while the "Nonflying Bird" subclass might include "Penguin" and "Kiwi." These specific bird classes inherit the general characteristics of the "Bird" class and also share attributes with their immediate parent class, whether it's "Flying Bird" or "Nonflying Bird."

Inheritance promotes code reuse. Instead of writing the same code multiple times, you can define common functionality in a base class and let derived classes inherit that functionality. This reduces redundancy and makes code easier to maintain.

Through inheritance, OOP supports polymorphism, where a single function or method can work with objects of different classes. For example, if a base class Shape has a method draw(), all subclasses like Circle, Square, and Triangle can override this method to provide their specific implementations. Yet, all these subclasses can be treated as instances of Shape, allowing for flexible and dynamic method invocation.

Inheritance makes it easy to extend existing code. You can create new classes based on existing ones and add or override specific functionalities. This allows developers to build complex systems that can be easily expanded and adapted without modifying existing code, ensuring that the system remains stable and maintainable.

Polymorphism: Polymorphism in Object-Oriented Programming (OOP), which is derived from the Greek word meaning "many forms/ multiple actions (Poly = Multiple, Morphing = Actions)." Polymorphism allows a single function or operator to behave differently based on the context, such as the type or number of arguments it receives i.e., the same operation can exhibit different behaviors depending on the types of data involved.

For example, the operation of addition (+) can perform different tasks depending on the operands: it can add two numbers to produce a sum, or if the operands are strings, it can concatenate them into a single string. This flexibility is a key feature of polymorphism in OOP and is often implemented through **operator overloading**, where operators are redefined to work with user-defined data types.

A specific type of polymorphism where a single function name is used to perform different tasks depending on the number or type of arguments passed to it is called as **function overloading**.

Figure 5 illustrates this concept: a base class Shape defines a function Draw(), which is then overridden in its derived classes like Circle, Box, and Triangle. Each of these derived classes provides its specific implementation of the Draw() function, allowing the same function name to handle different types of shapes.

This ability to use a single interface to represent different underlying forms is what makes polymorphism a powerful tool in OOP. It allows for more flexible and maintainable code by enabling objects to be treated as instances of their base class while still invoking behaviors specific to their derived class. Polymorphism enhances the capability of OOP systems to handle different data types and operations in a unified way, leading to more dynamic and adaptable code structures.

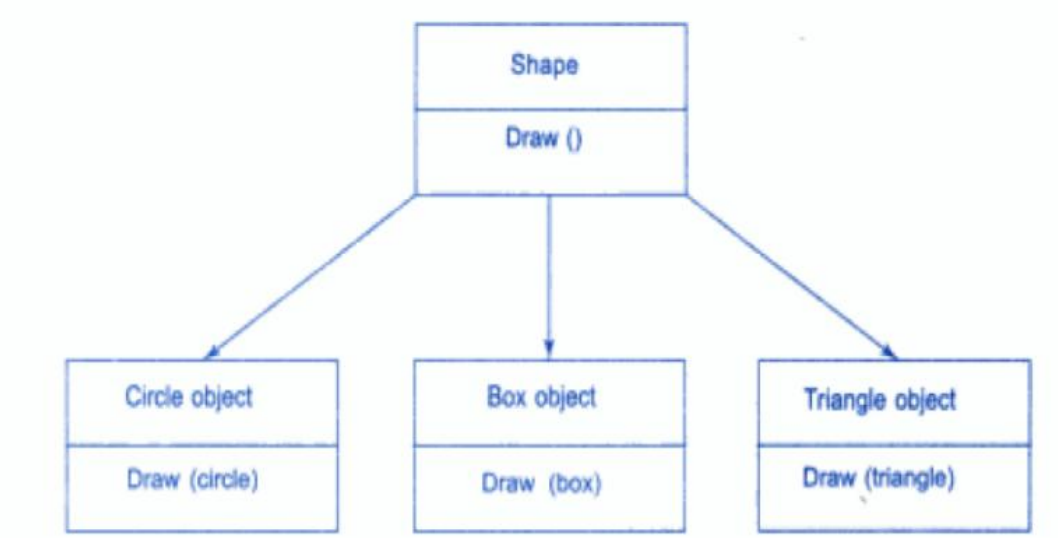

Figure 5: Polymorphism

Dynamic binding : in Object-Oriented Programming (OOP), it is also known as **late binding**. Binding, refers to the process of linking a procedure call to the specific code that will be executed in response to that call. Dynamic binding means that this linking process is deferred until runtime, rather than being determined at compile time. This allows the program to decide which specific piece of code to execute only when the program is actually running.

Dynamic binding is closely associated with polymorphism and inheritance. In polymorphism, a single function call might behave differently depending on the actual object it is associated with at runtime. This is useful when working with a class hierarchy where a base class reference can point to objects of any derived class. The specific method that gets called depends on the dynamic type of the object that the reference points to at the time of execution.

For example a draw() procedure in figure 5 illustrates dynamic binding. Imagine a base class that defines a draw() function, and various derived classes (like Circle, Box, Triangle) that each provide their own implementation of draw(). At runtime, when a draw() function is called on an object, dynamic binding ensures that the version of draw() specific to the object's actual type (e.g., Circle or Box) is executed. This flexibility allows for more dynamic and adaptable code, where the exact method that gets executed is determined based on the actual object in use at runtime, rather than being fixed at compile time.

Message passing: In Object-Oriented Programming (OOP), message passing is a fundamental aspect of how objects interact within a program. In an OOP system, the program is composed of a collection of objects, each of which is defined by a class. These objects communicate with each other by sending and receiving messages, similar to how people exchange information.

The process of programming in an object-oriented language typically involves three key steps:

- creating classes that define objects and their behaviours,
- instantiating objects from these class definitions, and
- establishing communication among the objects through message passing.

Message passing allows objects to request actions from each other, which is conceptually similar to sending a message in real life. When an object sends a message to another object, it is essentially making a request for the execution of a specific procedure or function within the receiving object. This involves specifying the name of the object to receive the message, the name of the function to be invoked (the message), and any necessary information or arguments required for the function to perform its task.

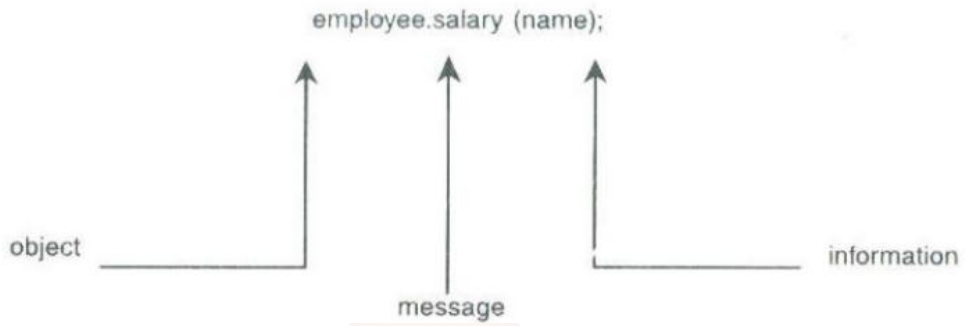

Figure 6: Message Passing

For example, in the figure 6, the message employee.salary(name) is shown as an illustration. Here, the employee object is sending a message to invoke the salary function with the argument name. This interaction results in the receiving object processing the request and returning the appropriate information.

The concept of message passing is important because it models real-world interactions, making it easier to build systems that simulate or reflect real-world scenarios. Objects in a program have a lifecycle—they can be created, interact through message passing, and be destroyed. As long as an object is alive, it can engage in communication with other objects, making message passing a dynamic and vital component of OOP.

3.1. Characteristics of OOP

Object-Oriented Programming (OOP) was developed as a response to the limitations found in procedural programming, where data and functions are often loosely coupled, leading to potential issues such as accidental data modification and poor scalability.

The key motivation behind OOP is to enhance data security and program structure by treating data as a fundamental element around which the program revolves. In OOP, data is closely tied to the functions that operate on it, encapsulating it within objects and preventing unauthorized or unintended modifications from other parts of the program. This encapsulation not only safeguards the data but also makes the program more modular and easier to manage.

The characteristics below collectively make OOP a powerful and flexible programming paradigm, especially for developing large and complex software systems.

- **Emphasis on Data Rather Than Procedure**: In OOP, the primary focus is on the data being manipulated, as opposed to the procedural approach where the sequence of actions or functions

is prioritized. OOP ensures that data is the central element around which functions revolve, safeguarding its integrity and ensuring that it is handled correctly within the program.

- **Programs are Divided into Objects**: OOP structures programs by breaking them down into smaller, manageable units called objects. Each object encapsulates both data and the methods (functions) that operate on that data, making the program modular and easier to manage.
- **Characterization of Objects Through Data Structures**: Objects are characterized by their data structures, meaning that the way data is organized within an object defines its nature and behavior. This design approach ensures that objects are self-contained entities that hold both the data and the operations that manipulate it.
- **Tying Functions with Data**: Functions that work on an object's data are bound together within the object's data structure. This tight coupling of data and methods ensures that the operations on the data are restricted to those defined within the object, leading to better data integrity and encapsulation.
- **Data Hiding**: OOP emphasizes the importance of hiding an object's internal data from external functions. This concept, known as encapsulation, prevents unauthorized access and modification of data, ensuring that it is only manipulated in controlled ways.
- **Object Communication via Functions**: While objects encapsulate their data, they are still capable of interacting with one another. This interaction happens through functions or methods, allowing objects to collaborate to achieve more complex behavior without directly exposing their internal data.
- **Ease of Adding New Data and Functions**: One of the strengths of OOP is its flexibility. New data and methods can be easily added to existing objects without disrupting the overall structure of the program. This makes it easier to extend and maintain software over time.
- **Bottom-Up Approach in Program Design**: OOP often follows a bottom-up approach in program design, where developers start by creating small, reusable objects and then combine them to build more complex systems. This contrasts with the top-down approach in procedural programming, where the focus is on defining the main functions and breaking them down into smaller tasks.

3.2. Benefits of Object-Oriented Programming (OOP)

The benefits of OOP are:

- **Eliminating Redundant Code**: Using inheritance, we can reuse existing code, which reduces the need to write the same code multiple times.

- **Building Programs Efficiently**: OOP allows us to build programs using standard modules that work together, saving time and improving productivity.
- **Improving Security**: Data hiding ensures that sensitive parts of a program are protected from being accessed or modified by other parts of the program.
- **Multiple Instances**: OOP makes it possible to create multiple copies (instances) of an object that can work simultaneously without affecting each other.
- **Mapping Real-World Problems**: It's easy to relate objects in a program to real-world problems, making the design more intuitive.
- **Work Partitioning**: OOP helps in dividing work among different parts of a project, making it easier to manage.
- **Detailed Design**: The focus on data allows us to create more detailed and accurate models of real-world problems.
- **Scalability**: Object-oriented systems can grow easily from small to large, adapting to more complex requirements.
- **Simplified Communication**: Message passing between objects makes it easier to manage interactions within the program and with external systems.
- **Managing Complexity**: OOP helps manage and reduce software's complexity, making it easier to develop and maintain.

4. DIFFERENCE BETWEEN C AND C++

C is a general-purpose programming language known for its flexible and compact structure. Programs written in C are typically composed of many small functions and focus on procedural programming. C++, derived from C, extends the language by introducing object-oriented features, allowing for both procedural and object-oriented programming.

The main difference between C and C++ is that C++ supports object-oriented programming (OOP), which emphasizes writing programs that are more modular, reusable, and maintainable. OOP allows for the packaging of data and functions into classes, promoting code reuse and better organization. In contrast, C supports only procedural programming, which is less modular and harder to maintain in large projects.

Key differences between C and C++ include:

- **Programming Approach**: C follows a top-down approach, while C++ supports a bottom-up approach, particularly in OOP design.
- **I/O Operations**: C uses printf and scanf for input/output, while C++ uses cin and cout for the same.
- **Function Declaration**: In C++, all functions must be declared before they are used, usually with prototypes. In C, functions can be used before being declared, although this is not considered good practice.
- **Structures**: C structures are purely data containers, while C++ structures can include member functions and access specifiers, making them more similar to classes.
- **Underscores in Identifiers**: In both C and C++, identifiers beginning with double underscores or containing them are reserved for compiler use and should be avoided.
- **Automatic Return in main()**: In C++, an explicit return 0; is not necessary in main() as the compiler automatically adds it, while in C, it is generally recommended to include it.

5. APPLICATIONS OF OBJECT-ORIENTED PROGRAMMING (OOP)

OOP has become widely popular among software engineers and is seen as a significant advancement in programming techniques. It is increasingly being used in various areas, especially in user interface design, such as in the development of windowing systems.

Specific Applications of OOP:

a. Real-Time Systems:OOP is beneficial for developing real-time systems that require quick and efficient processing of data and events.

b. Simulation and Modeling:OOP is used in simulation and modeling applications where complex systems need to be represented and manipulated.

c. Object-Oriented Databases:OOP concepts are applied to create and manage databases that use objects to represent data, enhancing the organization and retrieval of complex data structures.

d. Hypertext, Hypermedia, and Expert Text:OOP is utilized in creating hypertext systems, hypermedia applications, and expert systems, which require flexible and dynamic data handling.

e. AI and Expert Systems:Artificial Intelligence (AI) and expert systems benefit from OOP, as it helps in creating systems that can simulate human expertise and decision-making.

f. Neural Networks and Parallel Programming:OOP is instrumental in developing neural networks and parallel programming, where the ability to handle complex data and processes is crucial.

g. Decision Support and Office Automation Systems:OOP is used in building decision support systems and automating office tasks, improving efficiency and productivity.

h. CIM/CAM/CAD Systems: Computer-Integrated Manufacturing (CIM), Computer-Aided Manufacturing (CAM), and Computer-Aided Design (CAD) systems leverage OOP for designing, manufacturing, and integrating complex industrial processes.

6. SUMMARY

Let us recapitulate the important points discussed in this unit:

Computer performs a variety of tasks with the help of programming languages. In 1980, Bjarne Stroustrup, from Bell labs, began the development of the C++ language.

OOP is a bottom-up approach, focusing on objects that encapsulate both data and functions, which helps in organizing programs more effectively. In OOP, data is treated as a critical element, encapsulated within classes, thereby preventing it from being accessed or modified inadvertently. This encapsulation enhances data security and integrity. The core concepts of OOP include encapsulation, which ties data closely to the functions that operate on it within classes; objects and classes, where problems are solved by interacting objects that are instances of classes; and data hiding, which insulates data from direct access by other parts of the program. OOP also introduces data abstraction, which focuses on essential features while hiding unnecessary details, making complex systems easier to manage. Inheritance allows classes to inherit properties and behaviors from other classes, promoting code reuse. Polymorphism enables functions or methods to take on different forms, allowing the same function name to work with different types of data or objects. Dynamic binding allows the exact code to be determined at runtime, providing flexibility, while message passing facilitates communication between objects through method invocations.

The benefits of OOP are significant, with reusability being one of the most important advantages, allowing developers to write modular code that can be reused in different parts of a program or in different programs. OOP has gained widespread importance across various fields, particularly in real-time business systems.

7. SELF-ASSESSMENT QUESTIONS

Multiple choice Questions	
1	Which programming methodology uses a top-down approach? a) Object-Oriented Programming b) Procedural Programming c) Functional Programming d) Logic Programming
2	class is a _____? a) A function that operates on data b) A blueprint for creating objects c) A type of loop d) A method to manage memory
3	Which feature of OOP allows a class to inherit properties from another class? a) Encapsulation b) Abstraction c) Polymorphism d) Inheritance
4	Which language was the precursor to C++? a) Python b) Java c) C d) Fortran
5	What does encapsulation protect? a) Code reusability b) Data integrity c) Memory allocation d) Execution speed
6	Which of the following best describes polymorphism in OOP? a) One function can have multiple forms b) A class can have multiple parents c) Objects can only have one method

	d) A program can execute in multiple environments
7	What is the main focus of procedural programming? a) Data b) Objects c) Functions d) Inheritance
8	In OOP, an object's data is hidden and protected through which feature? a) Abstraction b) Inheritance c) Encapsulation d) Polymorphism
True or False:	
9	C++ supports both procedural and object-oriented programming.
10	Inheritance in OOP allows objects to inherit properties from multiple classes at the same time.
Fill in the Blanks:	
11	The concept of _____ in OOP refers to an object taking on different forms.
12	What is the term for a function defined within a class?

8. GLOSSARY

Programming Methodologies	-	The various approaches and techniques used in the process of software development. They include procedural programming, object-oriented programming, and others.
Procedural Programming	-	An early programming methodology that emphasizes a linear, top-down approach to writing programs, focusing on procedures or functions to operate on data.
Object-Oriented Programming (OOP)	-	A programming paradigm that uses "objects" to represent data and methods to model real-world entities and interactions, addressing the limitations of procedural programming.
Encapsulation	-	The bundling of data and the methods that operate on that data within a single unit or class, restricting direct access to some of an object's components.
Inheritance	-	A mechanism in OOP where a new class is derived from an existing class, inheriting its attributes and behaviors while allowing for additional customization.
Polymorphism	-	The ability of different classes to respond to the same function or method call in a way that is specific to their class type.
Abstraction	-	The process of hiding the complex implementation details of a system and exposing only the necessary and relevant features to the user.
Data Hiding	-	A principle in OOP where an object's internal state is protected from outside access, ensuring that only authorized methods can interact with its data.
Object	-	A self-contained entity that contains data (attributes) and methods (functions) that represent its behaviour.
Class	-	A blueprint or template for creating objects, defining the data and behaviour that the instantiated objects will have.
Method	-	A function that is defined within a class and can operate on the data contained within an object of that class.

Instance	-	A specific object created from a class.
Message Passing	-	The process by which objects communicate with each other in OOP by sending and receiving information through methods.

9. SAMPLE PROGRAMS

1. Simple C++ Program Without Using Classes

```
#include <iostream> // Modern header syntax
using namespace std;

int main()

{
   cout << "Welcome to C++ programming"; // Output to console
   return 0; // Indicate successful program termination
}
```

This program is a basic example of a C++ program that does not use any object-oriented concepts such as classes or objects. It simply includes the necessary header file for input and output operations and uses the main() function to print a message to the console. The cout object, which is part of the standard C++ library, is used to output the string "Welcome to C++ programming".

When this program is run, it will display the following output on the console:

```
Welcome to C++ programming
```

2. Program to add two integers

```
#include <iostream> // Modern header syntax for I/O
using namespace std; // Allows usage of standard library components
                     // without std:: prefix

int main()
{              // Execution starts at main() function

   int a, b, c;

   cout << "\nProgram to add 2 integer values";
   cout << "\nEnter the values of integers a & b: ";

   cin >> a >> b; // Read input values into a and b

   c = a + b; // Add the values of a and b, store in c

   // Output the values of a, b, and c
   cout << "\nThe value of a is " << a;
```

```
    cout << "\nThe value of b is " << b;
    cout << "\nThe sum (c) of a and b is " << c << endl;

    return 0; // Indicate successful program termination
}
```

The above C++ program is a simple example designed to demonstrate how to perform basic arithmetic operations, specifically the addition of two integers, and how to interact with the user through input and output operations. The program begins by including the necessary header file which is part of the C++ Standard Library. It provides facilities for input and output (I/O), like cin for input and cout for output.

However, it should be noted that in modern C++, iostream is used instead of iostream.h.

The program execution starts with the main() function, which is the entry point for any C++ program. Inside this function, three integer variables—a, b, and c—are declared. These variables are used to store the integers that the user will input (a and b) and their sum (c).

The program then uses the cout statement to display messages on the screen, informing the user about the program's purpose (to add two integers) and prompting them to enter the values for a and b. The cin statement is employed to capture the user's input, reading the values entered by the user and storing them in the variables a and b.

Once the input values are received, the program calculates the sum of a and b by using the expression c = a + b;, storing the result in the variable c. The program then outputs the values of a, b, and c using additional cout statements, displaying the input values and their calculated sum.

Finally, the program reaches the end of the main() function, at which point it terminates.

When the program is executed with inputs 5 and 10.

The output will be:

```
Program to add 2 integer values
Enter the values of integers a & b
5 10
The value of a is 5
The value of b is 10
 The sum (c) of a and b is 15
```

3. C++ Program to calculate the area of a rectangle

```
#include<iostream>  // Include the input-output stream library
using namespace std;

int main() {
   // Declare variables for length, width, and area
   double length, width, area;

   // Prompt the user to enter the length of the rectangle
   cout << "Enter the length of the rectangle: ";
   cin >> length;  // Read the length


   // Prompt the user to enter the width of the rectangle
   cout << "Enter the width of the rectangle: ";
   cin >> width;  // Read the width

   // Calculate the area of the rectangle
   area = length * width;

   // Output the area of the rectangle
   cout << "The area of the rectangle is: " << area << endl;

   return 0;  // Return 0 to indicate successful completion
}
```

The above C++ program is a straightforward example designed to calculate the area of a rectangle based on user input for the length and width. The program begins by including the iostream library, which is essential for handling input and output operations in C++. By using the using namespace std; directive, the program allows for easier access to standard library objects like cin and cout without needing to prefix them with std::. The program's execution starts with the main() function, where three variables—length, width, and area—are declared. These variables are of type double to accommodate the possibility of decimal values.

The program then prompts the user to enter the length and width of the rectangle using cout statements, and it captures these values using cin, storing them in the length and width variables. The area of the rectangle is calculated by multiplying the length and width, with the result stored in the area variable. After the calculation, the program outputs the computed area to the user with a clear

message indicating the result. The program concludes with a return 0; statement, signaling successful execution.

The output will be:

Enter the length of the rectangle: 5

Enter the width of the rectangle: 10

The area of the rectangle is: 50

4. C++ Program to Enter name and age and it will Display the Collected Information.

```
#include <iostream>  // Required for cin and cout
using namespace std;
int main() {
   int age;            // Variable to store the user's age
   string name;        // Variable to store the user's name
   // Prompt the user to enter their name
   cout << "Enter your name: ";
   cin >> name;        // Take input and store it in the variable 'name'
   // Prompt the user to enter their age
   cout << "Enter your age: ";
   cin >> age;         // Take input and store it in the variable 'age'
   // Display the collected information
   cout << "Hello " << name << "You are " << age << " years old." << endl;
   return 0;
}
```

This C++ program is designed to collect and display the user's name and age. It begins by including the <iostream> header file, which provides functionality for input and output operations using cin and cout. The using namespace std; statement allows the program to avoid prefixing standard library objects with std::.Inside the main() function, two variables are declared: name to store the user's name, and age to store the user's age. The program prompts the user to enter their name by printing the message "Enter your name: " to the console, followed by using cin to capture the user's input into the name variable.

Similarly, the program prompts the user to enter their age, storing the input in the age variable using cin. Finally, the program outputs a personalized message that includes the user's name and age using cout.

If the user's name is “Vijay” and age is 20. The program first prompts for the user's name and stores "Vijay" in the name variable. Then, it prompts for the user's age and stores "20" in the age variable. Finally, it displays the message "Hello Vijay You are 20 years old." by combining the stored values with the text in the cout statement.

The output will be:

```
Enter your name: Vijay
Enter your age: 20
Hello Vijay  You are 20 years old.
```

5. C++ Program : Celsius to Fahrenheit Conversion

```
#include <iostream>  // Required for cin and cout
using namespace std;
int main() {
   float celsius, fahrenheit;  // Declare variables for Celsius and Fahrenheit
   // Prompt the user to enter the temperature in Celsius
   cout << "Enter temperature in Celsius: ";
   cin >> celsius;  // Take input for Celsius temperature

   // Convert Celsius to Fahrenheit using the formula: (Celsius * 9/5) + 32
   fahrenheit = (celsius * 9/5) + 32;

   // Display the result
   cout << celsius << " Celsius is equal to " << fahrenheit << " Fahrenheit." << endl;
   return 0;
}
```

This C++ program converts a temperature from Celsius to Fahrenheit. It first prompts the user to enter a temperature in Celsius and stores the input in the variable celsius. Then, it uses the formula (Celsius * 9/5) + 32 to convert the value to Fahrenheit and stores the result in the variable fahrenheit. The program finally displays the conversion result by outputting a message in the format: "[celsius] Celsius is equal to [fahrenheit] Fahrenheit." If the user enters 25 as the Celsius :-

The output will be:

```
Enter temperature in Celsius: 25
25 Celsius is equal to 77 Fahrenheit.
```

10. TERMINAL QUESTIONS

1. Outline the evolution of programming methodologies from procedural programming to object-oriented programming.
2. What is polymorphism in OOP? Provide an example to illustrate your explanation.
3. Describe the process of message passing in OOP and its significance.
4. Explain the difference between abstraction and encapsulation in OOP.
5. What are the primary applications of Object-Oriented Programming? Give examples of fields where OOP is widely used.
6. Provide a detailed comparison between C and C++ focusing on their support for procedural and object-oriented programming.
7. Analyse the benefits and challenges of implementing Object-Oriented Programming in large-scale software projects.
8. Explain in detail the impact of OOP on software design and architecture, focusing on maintainability, scalability, and reusability.
9. Discuss the key characteristics of Object-Oriented Programming (OOP) in detail, with examples.
10. Explain how inheritance, polymorphism, and encapsulation work together in OOP to create modular and flexible software systems.

11. ANSWERS

Self-Assessment Questions

1. Procedural Programming
2. A blueprint for creating objects
3. Inheritance
4. C
5. Data integrity
6. One function can have multiple forms
7. Functions
8. Encapsulation
9. True
10. False
11. Polymorphism
12. Method

Terminal Questions Answers

1. Refer to section 1.2
2. Refer to section 1.3
3. Refer to section 1.3
4. Refer to section 1.3
5. Refer to section 1.5
6. Refer to section 1.4
7. Refer to section 1.3.2
8. Refer to section 1.3 and 1.3.1
9. Refer to section 1.3.1
10. Refer to section 1.3

12. REFERENCES

1. E Balagurusamy, “Object-Oriented Programming with C++”, 8th edition, 2020, Tata McGraw Hill Publications.
2. Object-Oriented Programming Using C++, By B. Chandra, CRC Press.
3. Starting Out with C++: From Control Structures Through Objects, Global Edition, by Tony Gaddis, Pearson Education.